

LAPORAN PRAKTIKUM
ALGORITMA DAN PEMROGRAMAN 1
MODUL 17
“SKEMA PEMROSESAN SEKUENSIAL”



DISUSUN OLEH:
RASYA PUTRA WIBOWO
109082500132
S1 IF-13-02
DOSEN:
Wahyu Andi Saputra, S.Pd., M.Eng

PROGRAM STUDI S1 TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2025/2026

DASAR TEORI

Pemrosesan sekuensial merupakan teknik pengolahan data yang dilakukan secara bertahap sesuai urutan data diterima. Setiap data diproses satu per satu hingga seluruh data selesai diolah. Pendekatan ini banyak digunakan karena sederhana, mudah diterapkan, dan tidak membutuhkan penggunaan memori yang terlalu besar.

Perulangan atau looping adalah mekanisme dalam pemrograman yang memungkinkan suatu blok perintah dieksekusi berulang kali. Dengan adanya perulangan, programmer tidak perlu menuliskan instruksi yang sama secara berulang sehingga program menjadi lebih ringkas dan efisien. Struktur ini sering dimanfaatkan untuk melakukan perhitungan, pengolahan data, pencarian, maupun simulasi.

Bilangan acak merupakan nilai yang dihasilkan komputer untuk merepresentasikan kondisi yang bersifat tidak pasti atau acak. Penggunaan bilangan acak sangat penting dalam berbagai simulasi, seperti percobaan statistik, pemodelan kejadian alam, permainan, serta berbagai kasus lain yang membutuhkan variasi data secara tidak terduga.

SOAL LATIHAN

Statement perulangan

1. Mencari Rerata

Source Code:

```
package main

import "fmt"

func main() {
    var angka float64
    var total float64
    var rata float64
    var jumlahData int

    fmt.Scan(&angka)

    for angka != 9999 {
        total += angka
        jumlahData++

        fmt.Scan(&angka)
    }

    if jumlahData != 0 {
        rata = total / float64(jumlahData)
        fmt.Printf("Rerata = %.2f\n", rata)
    } else {
        fmt.Println("Tidak ada data yang dihitung")
    }
}
```

Output:

```
PS C:\Users\ASUS\Downloads\Modul 17_ Rasya Putra Wibowo> go run rerata.go
10
20
30
40
9999
Rerata = 25.00
```

Deskripsi Program:

Program ini digunakan untuk menghitung nilai rata-rata dari sejumlah bilangan yang dimasukkan oleh pengguna secara berurutan. Proses input dilakukan terus-menerus hingga pengguna memasukkan angka **9999** sebagai penanda akhir data. Setiap bilangan yang dimasukkan akan dijumlahkan dan jumlah datanya akan dihitung, kemudian setelah proses selesai program akan menghitung rata-rata dengan membagi total seluruh bilangan dengan banyaknya data yang telah diinput. Jika tidak terdapat data yang dimasukkan sebelum angka 9999, program akan menampilkan pesan bahwa tidak ada data yang dapat dihitung. Program ini menerapkan konsep pemrosesan sekuensial dan perulangan dalam pengolahan data.

2. Mencari String

Source Code:

```
package main

import "fmt"

func main() {
    var kataCari, kata string
    var banyakData, indeks int
    var ada bool
    var letakPertama int
    var totalKemunculan int

    fmt.Print("Masukkan kata yang dicari: ")
    fmt.Scan(&kataCari)

    fmt.Print("Masukkan banyak data: ")
    fmt.Scan(&banyakData)

    ada = false
    letakPertama = -1
    totalKemunculan = 0

    for indeks = 1; indeks <= banyakData; indeks++ {
        fmt.Print("Masukan data ke-", indeks, ": ")
        fmt.Scan(&kata)

        if kata == kataCari {
            totalKemunculan++
        }
    }
}
```

```
        ada = true

        if letakPertama == -1 {
            letakPertama = indeks
        }
    }
}

fmt.Println()

fmt.Println("Hasil Pemeriksaan:")

if ada {
    fmt.Println("a. Kata yang dicari ditemukan dalam data.")
} else {
    fmt.Println("a. Kata yang dicari tidak ditemukan dalam data.")
}

if letakPertama != -1 {
    fmt.Println("b. Kemunculan pertama berada pada posisi", letakPertama)
} else {
    fmt.Println("b. Kata yang dicari tidak ditemukan.")
}

fmt.Println("c. Jumlah kemunculan kata =", totalKemunculan)

if totalKemunculan >= 2 {
    fmt.Println("d. Terdapat minimal dua kemunculan kata tersebut.")
}
```

```
    } else {  
  
        fmt.Println("d. Kemunculan kata kurang dari dua  
kali.")  
  
    }  
  
}
```

Output:

```
PS C:\Users\ASUS\Downloads\Modul 17_ Rasya Putra Wibowo> go run bilangan.go  
Masukkan kata yang dicari: mangga  
Masukkan banyak data: 5  
Masukan data ke-1: apel  
Masukan data ke-2: mangga  
Masukan data ke-3: mangga  
Masukan data ke-4: pir  
Masukan data ke-5: anggur  
  
Hasil Pemeriksaan:  
a. Kata yang dicari ditemukan dalam data.  
b. Kemunculan pertama berada pada posisi 2  
c. Jumlah kemunculan kata = 2  
d. Terdapat minimal dua kemunculan kata tersebut.
```

Deskripsi Program:

Program ini digunakan untuk mencari keberadaan suatu string dalam sekumpulan data string yang dimasukkan oleh pengguna. Pengguna terlebih dahulu memasukkan string yang ingin dicari serta jumlah data yang akan diperiksa, kemudian program akan membaca setiap data secara berurutan menggunakan proses perulangan. Selama proses berlangsung, program akan mengecek apakah data yang dimasukkan sama dengan string yang dicari, menghitung jumlah kemunculannya, serta mencatat posisi kemunculan pertama. Setelah seluruh data selesai diproses, program akan menampilkan informasi mengenai keberadaan string tersebut, posisi pertama ditemukan, jumlah kemunculannya dalam kumpulan data, dan apakah string tersebut muncul sedikitnya dua kali. Program ini menerapkan konsep pemrosesan sekuensial, pencarian data, dan perulangan dalam pengolahan input.

Pertanyaan:

A. Apakah String x ada dalam kumpulan n data string tersebut?

Jawaban untuk poin ini diperoleh dari variabel ditemukan. Jika selama proses pembacaan data terdapat string yang sama dengan x, maka nilai ditemukan akan menjadi true. Setelah seluruh data selesai dibaca, program dapat menentukan apakah string x ada atau tidak dalam kumpulan data.

B. Pada posisi ke berapa string x tersebut ditemukan?

Jawaban untuk poin ini diperoleh dari variabel posisi. Saat string x ditemukan pertama kali, indeks atau urutan data tersebut disimpan ke dalam variabel posisi. Dengan demikian program dapat menampilkan posisi pertama kemunculan string x pada kumpulan data.

C. Ada berapakah string x dalam kumpulan n data string tersebut?

Jawaban untuk poin ini diperoleh dari variabel jumlah. Setiap kali data yang dibaca sama dengan string x, nilai jumlah ditambah satu. Setelah seluruh data diproses, nilai jumlah menunjukkan banyaknya kemunculan string x dalam kumpulan data.

D. Adakah sedikitnya dua string dalam n data string tersebut?

Jawaban untuk poin ini diperoleh dengan memeriksa nilai jumlah. Jika jumlah bernilai 2 atau lebih, maka dapat disimpulkan bahwa terdapat sedikitnya dua string x dalam kumpulan data. Sebaliknya, jika jumlah kurang dari 2, maka syarat tersebut tidak terpenuhi.

3. Daerah Curah Hujan

Source Code:

```
package main

import (
    "fmt"
    "math/rand"
)

func main() {
    var jumlahTetes int
    var urutan int
    var koordinatX, koordinatY float64
    var wilayah1, wilayah2, wilayah3, wilayah4 int

    fmt.Print("Masukkan banyak tetesan hujan: ")
    fmt.Scan(&jumlahTetes)

    wilayah1 = 0
    wilayah2 = 0
    wilayah3 = 0
    wilayah4 = 0

    for urutan = 1; urutan <= jumlahTetes; urutan++ {
        koordinatX = rand.Float64()
        koordinatY = rand.Float64()

        if koordinatX < 0.5 && koordinatY < 0.5 {
            wilayah1++
        } else if koordinatX >= 0.5 && koordinatY < 0.5 {
            wilayah2++
        } else if koordinatX >= 0.5 && koordinatY >= 0.5
    {
        wilayah3++
    } else {
        wilayah4++
    }
    }

    fmt.Printf("Curah hujan wilayah 1: %.4f milimeter\n",
float64(wilayah1)*0.0001)
    fmt.Printf("Curah hujan wilayah 2: %.4f milimeter\n",
float64(wilayah2)*0.0001)
    fmt.Printf("Curah hujan wilayah 3: %.4f milimeter\n",
float64(wilayah3)*0.0001)
    fmt.Printf("Curah hujan wilayah 4: %.4f milimeter\n",
float64(wilayah4)*0.0001)
```

Output:

```
PS C:\Users\ASUS\Downloads\Modul 17_ Rasya Putra Wibowo> go run daerah.go
Masukkan banyak tetesan hujan: 10000000
Curah hujan wilayah 1: 250.1599 milimeter
Curah hujan wilayah 2: 249.9550 milimeter
Curah hujan wilayah 3: 249.7571 milimeter
Curah hujan wilayah 4: 250.1280 milimeter
```

Deskripsi Program:

Program ini digunakan untuk mensimulasikan distribusi curah hujan pada empat wilayah menggunakan bilangan acak. Pengguna memasukkan jumlah tetesan hujan yang akan disimulasikan, kemudian program akan menghasilkan pasangan koordinat acak yang mewakili lokasi jatuhnya setiap tetesan hujan pada suatu bidang. Berdasarkan posisi koordinat tersebut, setiap tetesan akan dikelompokkan ke salah satu dari empat wilayah yang berbeda dan jumlah tetesan pada masing-masing wilayah akan dihitung. Setelah seluruh simulasi selesai dilakukan, program menghitung dan menampilkan estimasi curah hujan pada setiap wilayah dalam satuan milimeter. Program ini menerapkan konsep perulangan, bilangan acak, simulasi, dan pemrosesan sekuensial dalam pengolahan data.

4. Mencari Formula

Source Code:

```
package main

import "fmt"

func main() {
    var nSuku int
    var k int
    var nilai float64
    var hasil float64

    fmt.Print("Masukkan jumlah suku: ")
    fmt.Scan(&nSuku)

    hasil = 0

    for k = 1; k <= nSuku; k++ {
        if k%2 != 0 {
            nilai = 1.0 / float64(2*k-1)
```

```

        } else {
            nilai = -1.0 / float64(2*k-1)
        }

        hasil += nilai
    }

    fmt.Printf("Nilai PI: %.7f\n", 4*hasil)

    k = 1
    hasil = 0

    for {
        if k%2 != 0 {
            nilai = 1.0 / float64(2*k-1)
        } else {
            nilai = -1.0 / float64(2*k-1)
        }

        hasil += nilai

        if (1.0 / float64(2*k+1)) <= 0.00001 {
            break
        }

        k++
    }

    fmt.Printf("Nilai PI: %.10f\n", 4*hasil)
    fmt.Println("Iterasi berhenti pada suku ke-", k)
}

```

Output:

```
PS C:\Users\ASUS\Downloads\Modul 17_ Rasya Putra Wibowo> go run formula.go
Masukkan jumlah suku: 1000000
Nilai PI: 3.1415917
Nilai PI: 3.1415726536
Iterasi berhenti pada suku ke- 50000
```

Deskripsi Program:

Program ini digunakan untuk menghitung pendekatan nilai π (pi) menggunakan deret Leibniz. Pengguna terlebih dahulu memasukkan banyaknya suku yang akan digunakan dalam perhitungan, kemudian program menjumlahkan suku-suku deret yang bernilai positif dan negatif secara bergantian untuk menghasilkan estimasi nilai π . Selanjutnya, program melakukan perhitungan ulang secara otomatis hingga nilai suku berikutnya lebih kecil atau sama dengan 0,00001 sebagai batas ketelitian yang ditentukan. Setelah proses selesai, program menampilkan hasil pendekatan nilai π serta jumlah iterasi yang diperlukan untuk mencapai tingkat ketelitian tersebut. Program ini menerapkan konsep pemrosesan sekuensial, perulangan, dan perhitungan numerik dalam menyelesaikan permasalahan matematika.

5. Banyak Topping

Source Code:

```
package main

import (
    "fmt"
    "math/rand"
)

func main() {
    var jumlahTopping int
    var nomor int
    var posisiX, posisiY float64
    var toppingDiPizza int
    var estimasiPi float64

    fmt.Print("Banyak Topping: ")
    fmt.Scan(&jumlahTopping)

    toppingDiPizza = 0

    for nomor = 1; nomor <= jumlahTopping; nomor++ {
        posisiX = rand.Float64()
        posisiY = rand.Float64()

        if (posisiX-0.5)*(posisiX-0.5)+(posisiY-
0.5)*(posisiY-0.5) <= 0.25 {
            toppingDiPizza++
        }
    }

    estimasiPi = 4.0 * float64(toppingDiPizza) /
float64(jumlahTopping)

    fmt.Println("Topping pada Pizza:", toppingDiPizza)
    fmt.Printf("PI : %.10f\n", estimasiPi)
}
```

Output:

```
PS C:\Users\ASUS\Downloads\Modul 17_ Rasya Putra Wibowo> go run kedai.go
Banyak Topping: 1234567
Topping pada Pizza: 969100
PI : 3.1398862921
```

Deskripsi Program:

Program ini mensimulasikan penyebaran topping pada sebuah pizza untuk memperkirakan nilai π (pi). Pengguna memasukkan jumlah topping yang akan dijatuhkan secara acak pada bidang persegi yang merepresentasikan area di sekitar pizza. Setiap topping yang jatuh di dalam area pizza berbentuk lingkaran akan dihitung sebagai topping pada pizza. Setelah seluruh topping diproses, program menghitung pendekatan nilai π berdasarkan perbandingan jumlah topping yang berada di dalam pizza terhadap jumlah seluruh topping yang dijatuhkan. Program ini menerapkan konsep simulasi, bilangan acak, perulangan, dan pemrosesan sekuensial.

DAFTAR PUSTAKA

- Munir, R. (2018). *Algoritma dan Pemrograman dalam Bahasa Pascal, C, dan C++*. Bandung: Informatika.
- Kurniawan, B. (2020). *Pemrograman Go untuk Pemula*. Jakarta: Elex Media Komputindo.
- Donovan, A. A., & Kernighan, B. W. (2016). *The Go Programming Language*. Boston: Addison-Wesley Professional.
- Gries, D., & Schneider, F. B. (1993). *A Logical Approach to Discrete Math*. New York: Springer.